

Technical Architecture Document

PrepFlow — Hybrid AI Exam Practice & Progress Tracker

1. Architecture overview

PrepFlow is a mobile-first Next.js web application. A secure backend owns scoring, data access, processing jobs, and AI calls. Question banks become permanent ordered database records. Theory books are privately stored and indexed for retrieval-based, persistent question generation.

Browser/PWA → Next.js UI and server routes → Supabase Auth/PostgreSQL/Storage + OpenAI Responses API/retrieval + background workers.

2. Recommended stack

Layer	Recommendation	Reason
Language	TypeScript	Type safety across UI and backend
Frontend	Next.js + React	Routing, server rendering, APIs in one codebase
Styling	Tailwind CSS + shadcn/ui	Fast accessible consistent UI
Database	Supabase PostgreSQL	Relational practice/analytics data
Auth	Supabase Auth	Email/password and secure sessions
Storage	Supabase Storage	Private original book files
AI	OpenAI Responses API	Structured outputs and source-grounded generation
Retrieval	OpenAI vector stores/file search	Semantic search over theory books
Jobs	Trigger.dev / Inngest / worker	Reliable async processing and retries
Validation	Zod	Validate API inputs and AI output
Charts	Recharts	Responsive trends and accuracy graphs
Hosting	Vercel + Supabase	Simple MVP deployment

3. Main modules

Module	Responsibility
Authentication	Signup, login, reset, session checks
Book upload	Validation, private storage, metadata
Document processing	Extraction, OCR detection, classification, section map
Question-bank engine	Ordered delivery and deterministic answer checking
Theory engine	Retrieval, AI generation, validation, persistence
Practice sessions	Attempt capture, timing, skip/unsure handling
Revision engine	Wrong/unsure scheduling and review tracking
Dashboard/analytics	Daily goals, trends, weak topics
Admin operations	Jobs, system health, consented support
Data lifecycle	Export and deletion cleanup

4. Project structure

app/(auth), app/(dashboard), app/admin, app/api; components/ui, dashboard, books, practice, revision, analytics; lib/supabase, openai, document-processing, practice, revision, analytics, security, validations; jobs; types; supabase/migrations; tests/unit, integration, e2e.

5. Core database schema

Table	Essential purpose/fields
profiles	id, full_name, role, timezone, onboarding_completed
exams	id, user_id, name, exam_date, daily_question_goal, is_active
books	id, user_id, title, storage_path, document_type, processing_status, vector_store_id
book_sections	book_id, sequence_order, title, section_type, subject, chapter, page range
questions	book_id, source_type, sequence_no, text, A-D options, correct_option, explanation, source refs
practice_sessions	user_id, exam_id, book_id, mode, start/end, counters
question_attempts	user_id, question_id, session_id, mode, selected_option, result, time, attempted_at
daily_goals	user_id, exam_id, local date, target, completed, is_completed
revision_queue	user_id, question_id, due_at, interval_days, review_count, status
book_processing_jobs	book_id, type, status, retry count, safe error message
audit_logs	actor, event type, resource, safe metadata, timestamp

6. Key relationships

profiles 1→many exams and books; books 1→many sections and questions; sessions 1→many attempts; questions 1→many attempts; attempts may create revision items; profiles 1→many daily goals.

7. Core algorithms

Next question: authenticate → verify book ownership/ready state → choose primary/revision/topic mode → for extracted bank fetch earliest unanswered valid sequence → for theory return already-saved next generated item or retrieve source, generate, validate, save transactionally, then return without answer key.

Score answer: authenticate → fetch owned question → validate A/B/C/D → compare on server with stored key → insert immutable attempt → update session/daily goal → schedule revision if incorrect/unsure → return result and explanation.

Generate theory question: retrieve only current book context → request schema-constrained output → validate four distinct options, valid key, source reference, no duplicate → save before showing user.

8. Environment variables

NEXT_PUBLIC_APP_URL; NEXT_PUBLIC_SUPABASE_URL; NEXT_PUBLIC_SUPABASE_ANON_KEY; SUPABASE_SERVICE_ROLE_KEY; OPENAI_API_KEY; OPENAI_PROJECT_ID; background-job credentials; SENTRY_DSN; optional rate-limit and email keys.

9. Configuration rules

- Never commit real environment files or credentials.
- Use only NEXT_PUBLIC variables in browser code.
- Keep OpenAI and Supabase service-role keys server-only.
- Use separate development, staging, and production projects.
- Calculate daily boundaries using stored user timezone.
- Use database transactions/RPC for answer submission so attempts, goals, and revisions stay consistent.